# Training-Time Optimization for Remote-Sensing Image Captioning on VRSBench

Technical report · Task 01: image captioning on VRSBench


## Abstract

We fine-tune Qwen2.5-VL-3B-Instruct with LoRA to generate detailed captions for VRSBench aerial and satellite images, on free 2× T4 GPUs. Six controlled runs on 2,000 training images (E0–E5) test where adapters should go, image resolution, learning-rate schedules and gradient accumulation. A final run (E8) combines the best choices with three efficiency and stability fixes: fp16 loss scaling, gradient accumulation and DistributedDataParallel. With these, the full 20,264-image training split trains in 136 minutes. Evaluated with BERT-BLEU$_4$ on 1,000 held-out images, the final model improves on the best 2,000-image model by +0.0073 [+0.0015, +0.0129]. Minimum-Bayes-Risk decoding adds a further +0.0093 [+0.0052, +0.0134], at 4.4× inference cost. Across all runs, the largest remaining loss is the length penalty (≈15% of the score), not caption content.


## 1. Task and data

**VRSBench** captions are long, human-verified descriptions of 512×512 Google Earth / DOTA / DIOR image tiles (mean 53 words in the train split, 48 words in our evaluation images). They name objects, counts and positions ("a small ship in the top-right").

| Split | Source | Images |
|---|---|---|
| Train | `VRSBench_train.json`, records whose question contains `[caption]` | 20,264 |
| Validation | `random.Random(2026).sample(VRSBench_EVAL_Cap, 200)`, used for every decision | 200 |
| Test | `random.Random(7).sample(remaining 9,150, 1000)`, used only for the final comparison | 1,000 |

E0–E5 train on a fixed random subset of 2,000 training captions (seed 42). E8 trains on all 20,264.

## 2. Model and architectural rationale

**Qwen2.5-VL-3B-Instruct** (3.75 B parameters):

- **Vision encoder.** A 32-layer ViT (hidden size 1280, patch 14) with 2D-RoPE. 28 layers use 112×112-pixel *window* attention (64 patches per window), so their cost grows linearly with image size. Only blocks 7, 15, 23 and 31 attend globally.
- **Merger.** A 2-layer MLP merges each 2×2 group of patches into one token, which cuts the number of visual tokens 4×.
- **Language model.** Qwen2.5-3B: 36 layers, hidden size 2048, grouped-query attention with 16 query heads and 2 KV heads (a KV cache of 36 KB per token in fp16), and multimodal RoPE (temporal, height and width position components).

**Why this model.** It accepts images at their native resolution instead of a fixed square crop, and its pre-training included grounding data with absolute coordinates. Both suit captions that describe small objects and their positions. At 3 B parameters it fits on one 16 GB T4 in fp16 (7.5 GB of weights), leaving room for activations. A dense 3 B model was preferred over Mixture-of-Experts models: an MoE model saves compute per token, but all of its experts must sit in GPU memory, and memory was the binding constraint here.

**Visual token budget.** `min_pixels = 224²` and `max_pixels = 448²`. A 512×512 tile is resized to 448×448, which gives 32×32 = 1,024 ViT patches and, after the 2×2 merge, **256 LLM tokens** (in general, H/28 × W/28).

## 3. Method

**Objective.** Next-token cross-entropy on the caption tokens only. The system prompt, the user prompt and the image tokens are masked with $-100$, and so is padding. The prompt ("Describe the image in detail.") and the ChatML template are identical in training and evaluation.

**LoRA.** The frozen weight $W_0$ is adapted as $W_0 + (\alpha/r)\cdot BA$, with $r = 16$, $\alpha = 32$ and dropout 0.05. B is initialised to zero, so training starts exactly from the pretrained model. Trainable parameters per target are $r\cdot(d_{in} + d_{out})$:

| Targets | Trainable | Share of 3.77 B |
|---|---|---|
| LLM q/k/v/o (36 layers) | 7,372,800 | 0.20% |
| + ViT `attn.qkv` / `attn.proj`, blocks 24–31 | 8,355,840 | 0.22% |
| + ViT, all 32 blocks | 11,304,960 | 0.30% |

These counts are read from the saved adapter files. Full fine-tuning with AdamW in mixed precision needs about 16 bytes per parameter, ≈ 60 GB. LoRA runs needed 10.8–14.3 GB at peak.

**Precision.** T4 GPUs have no bf16 support, so the base model runs in fp16. In E8 the LoRA weights are kept in fp32, and `torch.amp.GradScaler` scales the loss before backward. This stops small gradients from underflowing, and if a gradient overflows the scaler skips that one optimizer step instead of corrupting the weights.

**Decoding.** Captions are generated in batches with left padding, `max_new_tokens = 128`, and Qwen's default `repetition_penalty = 1.05`.

## 4. Evaluation protocol

The primary metric is **BERT-BLEU$_4$**:

$$P_n = \frac{1}{|R_n|} \sum_{r \in R_n} \max_{c \in C_n} \cos(E(c), E(r))$$

$$\text{BERT-BLEU}_4 = LP \cdot \exp\left(\tfrac{1}{4} \sum_{n=1}^{4} \log P_n\right)$$

$$LP = e^{-0.5|L_C - L_R|/L_R}$$

Implementation choices:

- E(·) is `bert-base-uncased`, last layer, each n-gram encoded on its own and mean-pooled over its word-pieces.
- $C_n$ and $R_n$ are n-gram sets.
- Lengths and n-grams use lower-cased word tokens.
- The score is computed per caption and then averaged.

Properties that matter for the analysis:

- Each $P_n$ is a semantic *recall*, while LP is symmetric: it penalises captions that are too short and too long.
- One word of length error costs ≈ 0.5 / 48 ≈ 1% of the score.
- The metric has a high floor. Random multilingual token strings scored 0.29–0.49, and the zero-shot model 0.40. Differences of ±0.01 are therefore meaningful.

All comparisons use a **paired bootstrap** (2,000–5,000 resamples of images, the same resample for every configuration) to obtain 95% CIs of scores and of differences. BLEU-4 (NLTK corpus BLEU, smoothing method 3) and BERTScore F1 (`roberta-large`, rescaled) are reported as secondary metrics on the 0–1 scale.

## 5. Experiments (2,000 training images)

All runs: 1 epoch, batch size 1, AdamW (weight decay 0.01), gradient clipping at 1.0, 1× T4, scored on the 200 validation images.

| Run | Change vs previous | BERT-BLEU$_4$ [95% CI] | Content (geo $P_{1\text{-}4}$) | LP | Pred. words | Train time | Throughput | Peak memory |
|---|---|---|---|---|---|---|---|---|
| E0 | LoRA on LLM attention, LR 5e-5 | 0.6586 [0.644, 0.672] | 0.780 | 0.846 | 46.5 | 19.5 min | 1.71 img/s | 10.8 GB |
| E1 | 324 visual tokens + gradient checkpointing | 0.6541 [0.638, 0.670] | 0.784 | 0.837 | 51.9 | 47.3 min | 0.71 img/s | 14.3 GB |
| E2 | ViT LoRA, last 8 blocks; LR 1e-5 (ViT) / 5e-5 (LLM) | 0.6625 [0.648, 0.676] | 0.780 | 0.851 | 47.0 | 21.3 min | 1.56 img/s | 11.1 GB |
| E3 | ViT LoRA, all 32 blocks | **0.6659** [0.652, 0.678] | 0.779 | 0.856 | 45.6 | 27.4 min | 1.22 img/s | 13.1 GB |
| E4 | E3 + linear warmup (100 steps) and decay | 0.6600 [0.644, 0.675] | 0.785 | 0.843 | 50.7 | 23.2 min † | 1.22 img/s | 13.7 GB |
| E5 | E3 + gradient accumulation 2 | 0.6607 [0.646, 0.675] | 0.784 | 0.844 | 50.0 | 27.0 min | 1.23 img/s | 13.3 GB |

† E4 hit a non-finite fp16 loss at step 1,690 and stopped there. Its logged train loss of 1.376 is an artifact: the loss summed over 1,690 steps was divided by 2,000. E6 (2 epochs) failed the same way, and E7 (LoRA rank 32) showed no consistent gain.

### Observations

- **Resolution (E1).** The $504^2$ input gives 1,296 patches, so the four global-attention ViT layers cost $(1296/1024)^2 \approx 1.6\times$ and the windows no longer tile evenly. Training took 2.4× longer and used +3.4 GB, with no quality gain. The 256-token budget was kept for all later runs.
- **Vision adaptation (E2, E3).** Adapting more ViT blocks raised BERT-BLEU$_4$, BLEU-4 (0.088 → 0.096 → 0.098) and ROUGE-L. Overhead imagery is far from the natural-image pretraining data, so the vision side has to move too. Its LR is 5× lower than the LLM's, because the ViT was aligned to the LLM during pre-training. Cost: +41% time and +2.3 GB.
- **Schedule and accumulation (E4, E5).** These gave no gain at this scale. With batch size 1, 2,000 steps and a constant LR, E3 was not schedule-limited. Gradient accumulation does not change throughput, because the step time is dominated by the forward and backward passes.
- **No run is significantly better than E0 on 200 images.** The confidence intervals are about ±0.013 wide. This is why the final comparison uses 1,000 held-out images.

## 6. Final training run: E8

E8 is E3's configuration plus the changes below, trained on all 20,264 images:

| Change | Reason |
|---|---|
| Effective batch 8: 1 image × 4 accumulation steps × 2 GPUs | Batch 1 gives very noisy gradients; gradient variance ∝ 1/batch |
| LR 1e-4 (LLM) / 2e-5 (ViT), 3% warmup, cosine decay | 8× fewer optimizer steps than at batch 1, so a higher LR, decayed to 0 |
| GradScaler, fp32 LoRA weights | Prevents the E4/E6 fp16 failures |
| DistributedDataParallel via `torchrun`; `no_sync()` on the 3 of 4 micro-steps between optimizer updates | Both GPUs used; one gradient all-reduce per optimizer step |
| 2 DataLoader workers | Images are decoded and pre-processed while the GPU computes |
| Adapter snapshots at 25/50/75/100% | A data-scaling curve from a single run |

| Metric | E3 (1× T4, 2k images) | E8 (2× T4, 20,264 images) |
|---|---|---|
| Wall-clock | 27.4 min | 135.8 min (an identical earlier run: 130.0 min) |
| Throughput | 1.22 img/s | 2.49 img/s (2.60 in the earlier run), ≈ 1.25 img/s per GPU |
| Peak memory | 13.1 GB | 12.7 GB per GPU |
| Stability | E4/E6 crashed in fp16 | finished; 4 of 2,533 steps skipped by the scaler (8 in the earlier run) |
| Train loss | 1.59 (epoch mean) | 2.33 at the start → ≈ 1.37 at the end (logging-window means) |

Per-GPU throughput is the same as on a single GPU, so DDP scaled almost linearly. Part of the gain over E3 also comes from the DataLoader workers: E3 loaded images in the main process.

**Data scaling (greedy, 200 val images):**

| Images seen | 5,066 | 10,132 | 15,198 | 20,264 |
|---|---|---|---|---|
| BERT-BLEU$_4$ | 0.6702 | 0.6699 | 0.6721 | 0.6700 |

E3 scores 0.6672 here. On validation the score is flat from 5k images on, but BLEU-4 rose from 0.104 to 0.112 and the content term rose too. The checkpoints were also taken at different points of the cosine schedule, so this curve mixes "more data" with "lower LR".

## 7. Decoding and inference study (200 validation images)

| Adapter | Decoding | BERT-BLEU$_4$ | LP | BLEU-4 | Words | s/image |
|---|---|---|---|---|---|---|
| none (zero-shot) | greedy | 0.3998 | 0.530 | 0.027 | 103.6 | 1.26 |
| E3 | greedy | 0.6672 | 0.858 | 0.098 | 45.5 | 1.03 |
| E3 | beam 4, length_penalty 0.6 / 1.0 / 1.4 | 0.6503 / 0.6597 / 0.6584 | 0.843 / 0.850 / 0.838 | 0.089 / 0.097 / 0.101 | 34 / 39 / 51 | 2.4–2.7 |
| E3 | MBR, greedy + 5 samples (T = 0.7, top-p = 0.9) | 0.6758 ‡ | 0.868 | 0.089 | 46.6 | 4.66 |
| E4 | greedy / beam 4 / MBR | 0.6594 / 0.6574 / 0.6579 | 0.842 / 0.840 / 0.841 | 0.094 / 0.112 / 0.087 | 51 / 46 / 52 | 1.1 / 2.7 / 5.1 |

‡ An earlier MBR run without a fixed sampling seed scored 0.6722: sampling noise is about ±0.004.

- **Fine-tuning mostly fixes length and style.** Zero-shot captions are 2.2× too long (LP 0.53). After fine-tuning, LP is 0.86, while the content term moves less (0.752 → 0.779).
- **Beam search lowers BERT-BLEU$_4$ but can raise BLEU-4** (E4: 0.094 → 0.112). Beam favours high-probability, shorter and more generic phrasing. That improves exact n-gram precision but lowers semantic recall and LP. The decoding strategy has to match the target metric.
- **MBR** picks the candidate with the highest average BERT-BLEU$_4$ against the other five. It uses no reference, so it is a legitimate decoding method. Outlier lengths lose on LP, so MBR also improves length matching.
- **Inference engineering.**
- Captions are generated in batches of 8 images (at most 16 sequences per call for beam search and sampling).
- On CUDA out-of-memory, a batch is split in half and retried.
- `remove_invalid_values` stops NaN logits from crashing the sampler.
- LoRA adds no latency once merged into the base weights.

## 8. Final results (1,000 held-out test images)

| Model | Decoding | BERT-BLEU$_4$ [95% CI] | Δ vs E3 greedy (paired 95% CI) | Content | LP | BLEU-4 | Words (ref 48.4) | s/image |
|---|---|---|---|---|---|---|---|---|
| E3 | greedy | 0.6694 [0.663, 0.675] | n/a | 0.781 | 0.858 | 0.097 | 46.0 | 1.07 |
| E8 | greedy | 0.6767 [0.670, 0.683] | +0.0073 [+0.0015, +0.0129] | 0.789 | 0.858 | 0.110 | 45.4 | 1.09 |
| E8 | MBR | **0.6860** [0.680, 0.692] | +0.0166 [+0.0114, +0.0220] | 0.792 | 0.867 | 0.105 | 47.1 | 4.73 |

E8 MBR vs E8 greedy: +0.0093 [+0.0052, +0.0134]. MBR kept the greedy caption for 40% of images, scored higher on 37% and lower on 24%. Length correlation with the reference rose from 0.20 to 0.26.

- **Full-data training improved content, not length.** Content rose from 0.781 to 0.789 and BLEU-4 by 14% relative, while LP is unchanged at 0.858. The 200-image validation set could not resolve this difference; the 1,000-image test set could.
- **MBR improved mostly length** (LP 0.858 → 0.867) at 4.4× inference time.

**Robustness check.** Batched fp16 generation produced 9 broken captions out of ~5,800 (`!!!!…` or random tokens) on 5 distinct images, 2–3 per test run. Excluding the 4 affected test images (996 remain), the conclusions hold:

- E8 − E3: +0.0067 [+0.0015, +0.0119]
- MBR − greedy: +0.0086 [+0.0047, +0.0125]
- E8 MBR − E3: +0.0153 [+0.0105, +0.0200]

## 9. Problems found and fixed

| Problem | Symptom | Cause | Fix |
|---|---|---|---|
| fp16 overflow in training | E4/E6: `Non-finite loss`, run stops | No loss scaling on a GPU without bf16 | GradScaler + fp32 LoRA weights; a step is skipped on overflow; the run stops after 20 skips in a row |
| Misleading train loss | E4 reported 1.376 | Sum over 1,690 steps divided by 2,000 | Mean over the steps actually run, logged per window |
| Wrong ids in saved predictions | `image_id` ≠ image of the reference | Saved by zipping `val_records` instead of `val_subset` | Ids restored from the seeded subset (`results/experiments/val_subset_200.json`) |
| Sampler crash at test time | `device-side assert triggered` during MBR | NaN logits from fp16 overflow reach `multinomial` | `remove_invalid_values=True` |
| Broken captions in batched generation | `!!!!…` captions | NaN logits with left-padded fp16 batches; greedy `argmax` picks token 0 | Detect, regenerate that image alone (then with an fp32 ViT); MBR ignores broken candidates |
| GPU memory never released | OOM on the first batch, 15 GB held at 0% utilisation | Model-loading cell re-run, stacking copies | Loading cell frees the previous model first and reports memory per GPU |
| Unreproducible MBR | 0.6722 vs 0.6758 across runs | No sampling seed | Seed per batch |

## 10. Reproducibility

Every number above comes from a notebook in `notebooks/` and is stored in `results/`. Seeds: training subset 42, validation subset 2026, test subset 7, bootstrap 0. The validation image ids are in `results/experiments/val_subset_200.json`. All generated captions are in `results/decoding_eval/predictions/`, and the LoRA adapters of E3 and E8 are in `adapters/`, so every reported score can be recomputed without training.

## 11. Limitations and next steps

- Length remains the largest loss (−15% via LP), and the model barely adapts caption length to image content. Next steps would be length-aware training (conditioning on a length bucket, or reweighting the end-of-sequence token) or MBR with more candidates.
- The validation set (200 images) is too small to rank runs that differ by less than ~0.01. The test set fixes this for the final comparison only.
- E8 changes several things at once (data, batch size, LR, schedule). Its gain is measured end to end; the data-scaling curve isolates the data effect only partially.
- fp16 inference is fragile on T4. Hardware with bf16 would remove both the loss-scaling and the broken-caption issues.